

Reinforcement learning

Agent and environment, return and discount, Q values, the Bellman equation and value iteration.

THE QUESTION

An agent's actions pay off, or cost it, many steps later, and nobody labels the right action. How can it learn good choices from trial and error alone?

THE INTUITION

Score every action in every state by the discounted reward that follows it: $Q(s, a)$. The Bellman equation ties each score to the best score one step later, so each step of experience can nudge one entry toward "reward now plus discounted best next".

KEY TAKEAWAY

Act by $\arg \max_a Q(s, a)$ but explore with probability ϵ . Move $Q(s, a)$ an α fraction of the way to $r + \gamma \max_{a'} Q(s', a')$. Continuous states need bins or a network.

observe state s → pick a: ϵ -greedy → environment: r, s' → update $Q(s, a)$ → $s \leftarrow s'$ → until the episode ends

1 The agent and the environment take turns

Flappy Bird (slides)
actions tap, no tap
reward score
state distances to pipe, ground

Mountain car (notebooks)
actions 0 left, 1 none, 2 right
reward -1 every step
state (position, velocity)

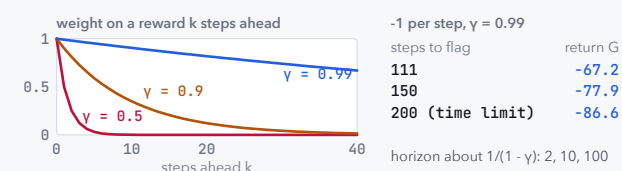
In state s_t the agent picks action a_t . The environment (the game's code and physics) returns a reward r_t and an observation, which becomes the next state s_{t+1} ; it depends on the current state and action. An episode runs from a reset until the game ends or a step limit. Gym: `env.step(a)` returns `obs, r, terminated, truncated, info`.

2 Mountain car: the engine is too weak to climb



Position is in $[-1.2, 0.6]$, velocity in $[-0.07, 0.07]$. The episode ends at the flag or after 200 steps, and each step costs -1 . Always pushing right only rocks the car between -0.45 and -0.33 . Backing up left for 50 steps first reaches the flag after 114 steps: the best action now depends on what comes later.

3 The return discounts later rewards



The agent maximises the return $G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots = \sum_{k \geq 0} \gamma^k r_{t+k}$ with $0 < \gamma < 1$, so $G_t = r_t + \gamma G_{t+1}$. A reward k steps away counts γ^k . With -1 per step for T steps, $G = -(1 - \gamma^T)/(1 - \gamma)$: reaching the flag sooner gives a higher return.

4 $Q(s, a)$ is the expected return of an action

Q table from the slides

state	action 1	action 2	action 3
S1	10	20	15
S2	20	30	5
...			
SN	-5	10	20

read one row
in S1: action 2 (20)
in S2: action 2 (30)
in SN: action 3 (20)
 $V(S2) = \max: 30$
greedy policy: argmax of the row

$Q(s_t, a_t) = \mathbb{E}[G_t | s_t, a_t]$: the return expected after taking a_t in s_t and acting well afterwards. With few states it is a table, a row per state and a column per action. Acting is a lookup, $\pi(s) = \arg \max_a Q(s, a)$, and $V(s) = \max_a Q(s, a)$. Learning fills the table in.

5 The Bellman equation links a state to the next

our grid, $\gamma = 0.9$

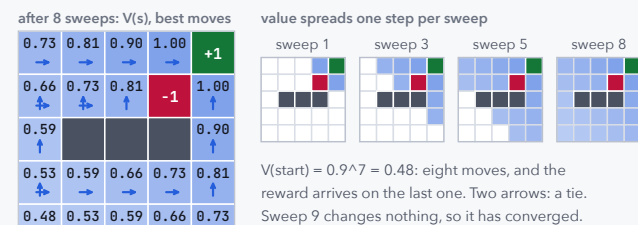
$Q(s, a) = r + \gamma V(s')$ at the ringed cell

action	reward	next state	$Q(s, a)$
up: into goal	1 + 0	(1,3)	1.00
right: wall, stay	0 + 0.9 × 1.00	(1,1)	0.90
down	0 + 0.9 × 0.90	(1,2)	0.81
left: into pit	-1 + 0	(1,0)	-1.00

$V(s) = \max = 1.00$: go up. After the goal or pit: no future term.

$Q(s, a) = R(s, a) + \gamma \max_{a'} Q(s', a')$: reward now plus the discounted best value one step on; for values, $V(s) = \max_a [R(s, a) + \gamma V(s')]$. It assumes each action has one outcome; with random outcomes, average over s' . Our grid: walls dark, goal +1 and pit -1 (both end the episode), 0 for other moves.

6 Value iteration applies Bellman until nothing changes



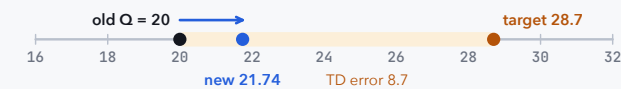
When the rules (R and s') are known, start from $V = 0$ and apply the right-hand side to every state, sweep after sweep. Values shrink by γ per step from the goal (1, 0.9, 0.81, ...) and the argmax arrows are the best moves. Reinforcement learning handles unknown rules: the agent can only act and watch.

Q-learning, exploration and large state spaces

One update by hand, the loop in code, exploration, binned states and deep Q-networks.

7 Q-learning moves Q part way to the target

in S1 take action 2, get $r = -1$, land in S2 (slides' table, $\alpha = 0.2$, $\gamma = 0.99$)



$$\text{target} = -1 + 0.99 \times 30 = 28.7; Q = 20 + 0.2 \times 8.7 = 21.74$$

Without the rules, use each step (s, a, r, s') as a sample of the Bellman right side:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$

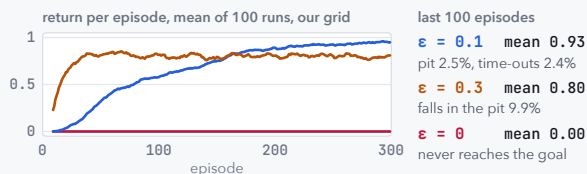
The bracket is the temporal-difference (TD) error. $\alpha = 1$ copies the target; the notebook uses 0.2. At an episode's last step the target is just r .

8 The Q-learning loop in NumPy

```
for ep in range(n_episodes):
    s, done = reset(), False
    while not done:
        if rng.random() < eps: # explore
            a = rng.integers(n_actions)
        else: # exploit
            a = np.argmax(Q[s])
        s2, r, done = step(s, a)
        future = 0 if done else Q[s2].max()
        td = r + gamma * future - Q[s, a]
        Q[s, a] += alpha * td
        s = s2
```

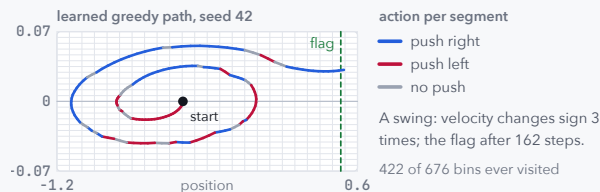
No model of the environment is needed. Q-learning is **off-policy**: the target uses the best next action even while exploring.

9 ϵ -greedy trades exploring for exploiting



With probability ϵ take a random action, otherwise the argmax. With $\epsilon = 0$ and $Q = 0$ all actions tie, np.argmax picks the first (up), and the agent bumps the top wall until the 50-step limit: no reward ever breaks the tie. $\epsilon = 0.3$ learns first but falls in the pit more; $\epsilon = 0.1$ ends higher.

10 Continuous states are cut into bins



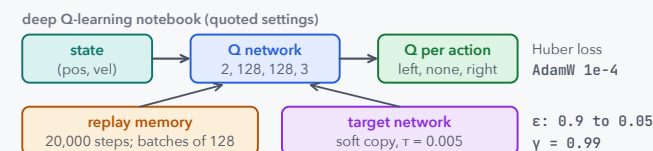
A table needs discrete states. `np.digitize` returns bin indices: with edges $(-1.2, -0.3, 0.6)$ it maps $-1.3, -1.2, -1.1, 0.6, 5.0$ to $0, 1, 1, 2, 3$. The notebook's 25 edges per axis make 26 bins, so 676 states times 3 actions. Coarse bins merge states that need different actions; fine bins leave many unvisited.

11 Q-learning gets the car to the flag



Our re-run copies the notebook: $\alpha = 0.2$, $\gamma = 0.99$, random-normal starting Q , $\epsilon = 0.8 \times 0.98^{\text{episode}}$ (under 0.002 by episode 300). Later exploration comes from the starting values: near 0, above the true Q values (-1 to -100 here), so untried actions look good. Random draws differ, so counts differ.

12 Deep Q-learning replaces the table



A network maps a state to one Q value per action and regresses the TD target. The deep notebook adds a **replay memory** (random minibatches of past steps, which breaks the correlation between consecutive steps) and a slowly updated **target network** for $\max Q(s', \cdot)$. The quick version in the Q-learning notebook has neither; its printed loss jumps to 26,067.

13 Pause and think

Rewards 0, 0, 1, $\gamma = 0.9$: G_0 ? 0.81.
 $\alpha = 1$: new Q ? The TD target.

Horizon of $\gamma = 0.99$? About 100.
Entries, $26 \times 26 \times 3$? 2028.

$P(\text{argmax})$, 3 actions, $\epsilon = 0.3$? 0.8.
Target network? Steadier targets.

agent acts, environment returns $r, s' \cdot G = \sum \gamma^k r \cdot Q = r + \gamma \max Q(s', \cdot) \cdot \text{move } \alpha \text{ toward the target} \cdot \text{explore with } \epsilon$

Before: the K-nearest neighbours sheet; next: the constrained optimisation and KKT sheet. Source: lecture slides "Reinforcement learning" and the gym environments, Q-learning and deep Q-learning notebooks (nipunbatra.github.io/ml-teaching). Mountain car is re-implemented in NumPy and matches the notebooks' printed runs; the grid world is ours (illustrative). Corrected: in the slides' return $\sum_{i=t}^{\infty} \gamma^i r_i$ the weight should be γ^{i-t} ; 25 bin edges need a 26×26 table, where the Q-learning notebook allocates 25×25 .